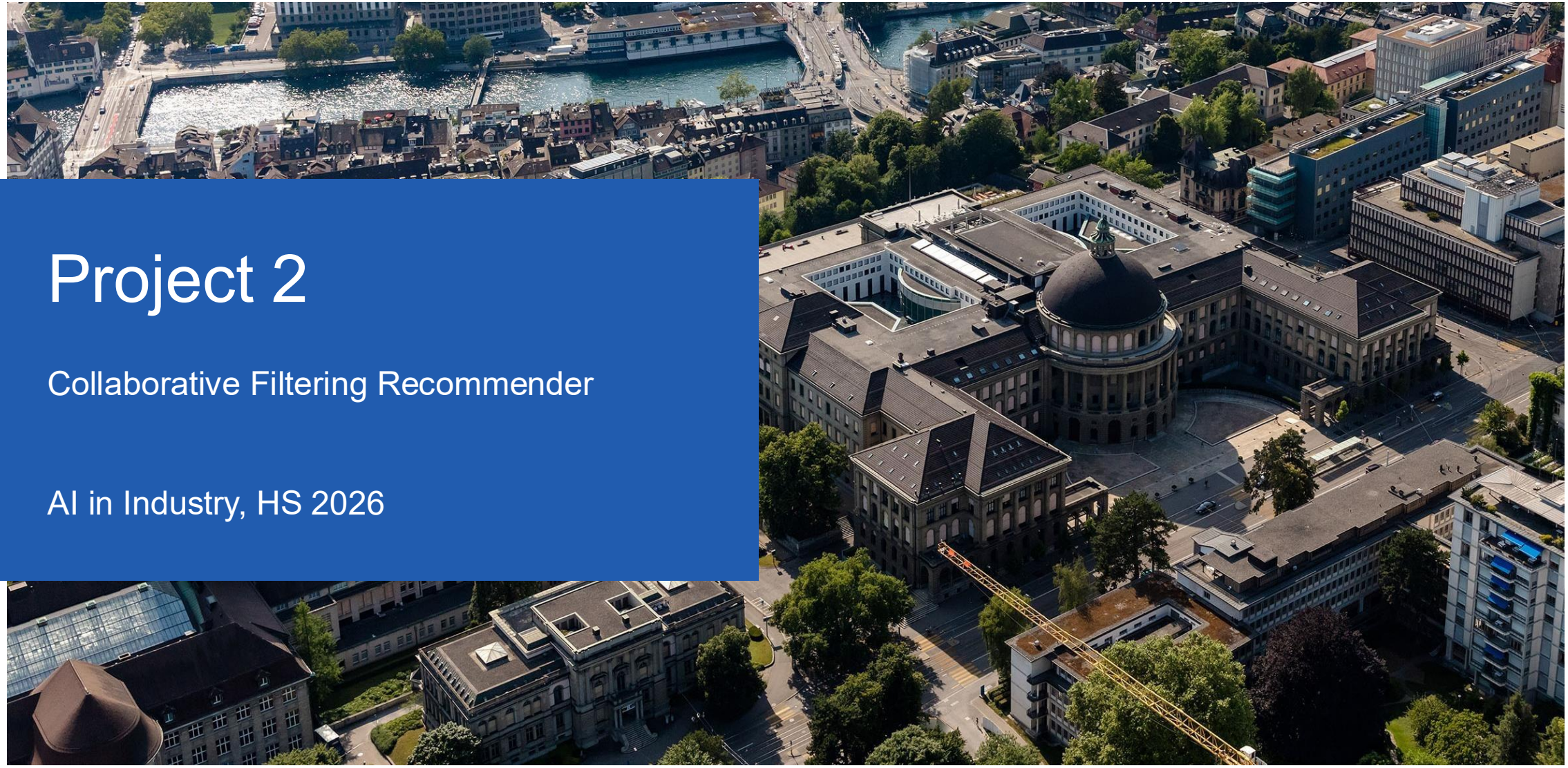


# Project 2

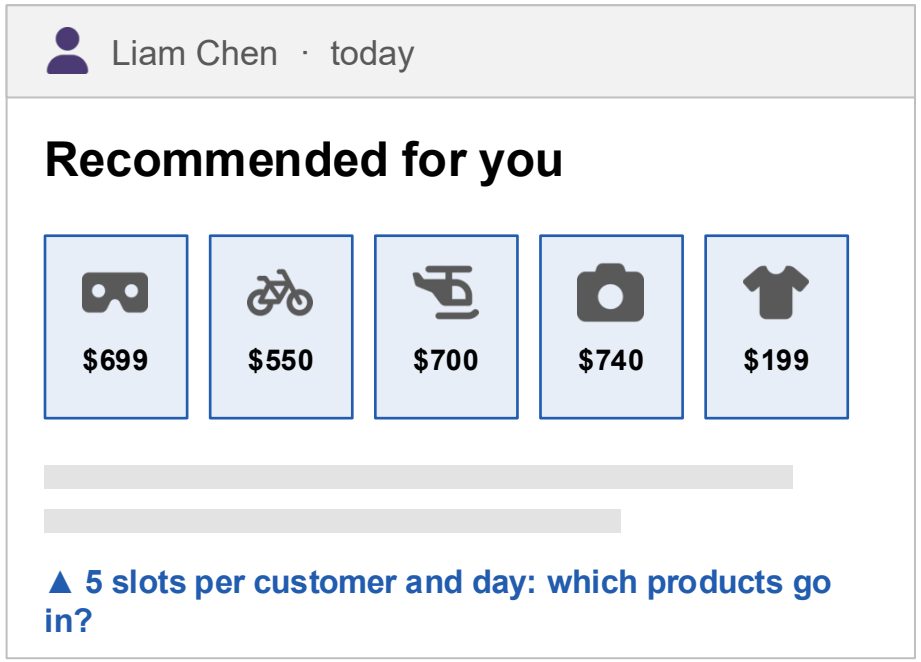
## Collaborative Filtering Recommender

AI in Industry, HS 2026

ETH zürich

# Motivation

# Project: AI product recommender




- You run the recommender of an **online shop** with **40 customers** and **50 products**.
- Every day the shop shows each customer **K = 5 products**. Each one is bought or not.
- You build a **collaborative filtering recommender** that picks the products that ***maximize the expected profit***.
- It learns only from **purchase feedback**: what was bought, and what was shown but not bought.

# Challenges



**Different customers, different tastes**

An architect with a high income and a customer on a tight budget do not buy the same products.



**You cannot read their minds**

You only see what was bought, not why, and never the true purchase probability.



**Cold start**

In training each customer sees only 15 of the 50 products. For the other 35 you have to guess.

**Idea:** Learn from past purchases to predict how likely each customer is to buy each product. Customers who buy alike help each other, so the model can also guess for products a customer has never seen.

ETH zürich

# The World

Simulation of an online shop

# What is in the World?



**Customers**

40 customers with age, job, income, family status and segment.



**Products**

50 products from $12 to $740, each with a price and a cost.



**Hidden purchase probability**

For every customer–product pair a secret $\kappa(c,i)$: how likely the customer buys it.



**Recommender agent**

Shows 5 products per customer a day and learns from the outcome. **You build this.**

Everything is **simulated**, so you can run 100 shopping days in a few minutes.

# Customers and products

Every customer and every product is one **record** in the database:

### Customer

```
{
    'name': 'Liam Chen',
    'age': 45,
    'job_type': 'Architect',
    'income': 250000,
    'family_status': 'Married',
    'segment': 'H'
}
```

### Product

```
{
    'name': 'Nexus VR Gaming Headset',
    'price': 699,
    'cost': 529,
    'description': 'Immersive ...'
}
```

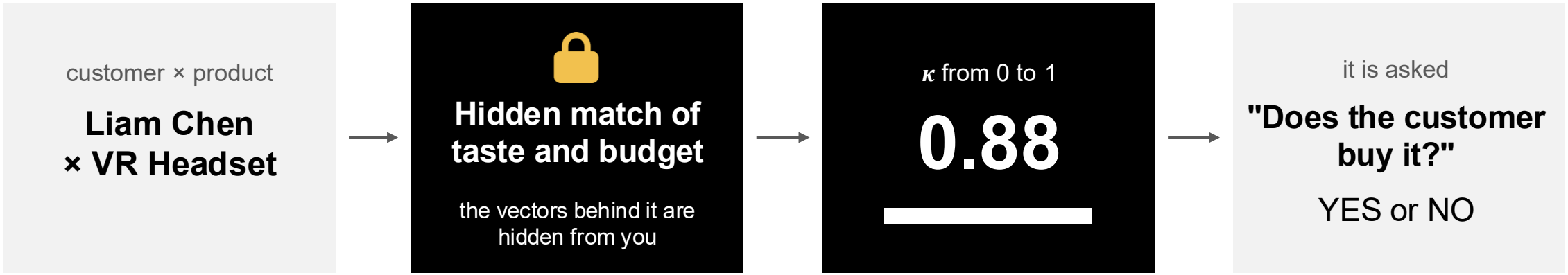
**40**
customers

**50**
products

**2,000**
customer–product pairs

**margin**
= price − cost

# Simulation: hidden purchase probability




The same customer, five different products:

VR Headset **0.88** Thermostat **0.82** Silk Scarf **0.23**

Leather Tote **0.05** T-Shirt **0.01**

For you it is a **black box**: you only see whether a product was bought, never $\kappa$ itself.

# What you see, and what stays hidden

## You see

- The **customer data**: 40 records
- The **product data**, with price and cost
- Which **5 products** were shown to whom
- Which of them were **bought** (implicit feedback)

## Hidden

- The true purchase probability $\kappa(c,i)$ for all 2,000 pairs
- The **taste and budget** vectors behind it
- Which 5 products would be the **best ones**

# The shopping day cycle

1

## The shop recommends

Every customer gets the 5 products with the highest expected profit.

Top 5

2

## The customer looks

Every shown product has its hidden $\kappa$ for this customer.

κ(c,i)

3

## Each product flips a biased coin

Bought with probability $\kappa$: likely, but no guarantee.

Purchase

4

## Learn from the session

Bought: push $\hat{\kappa}$ up. Shown but not bought: push $\hat{\kappa}$ down.

Update

**expected profit = margin × $\hat{\kappa}$**

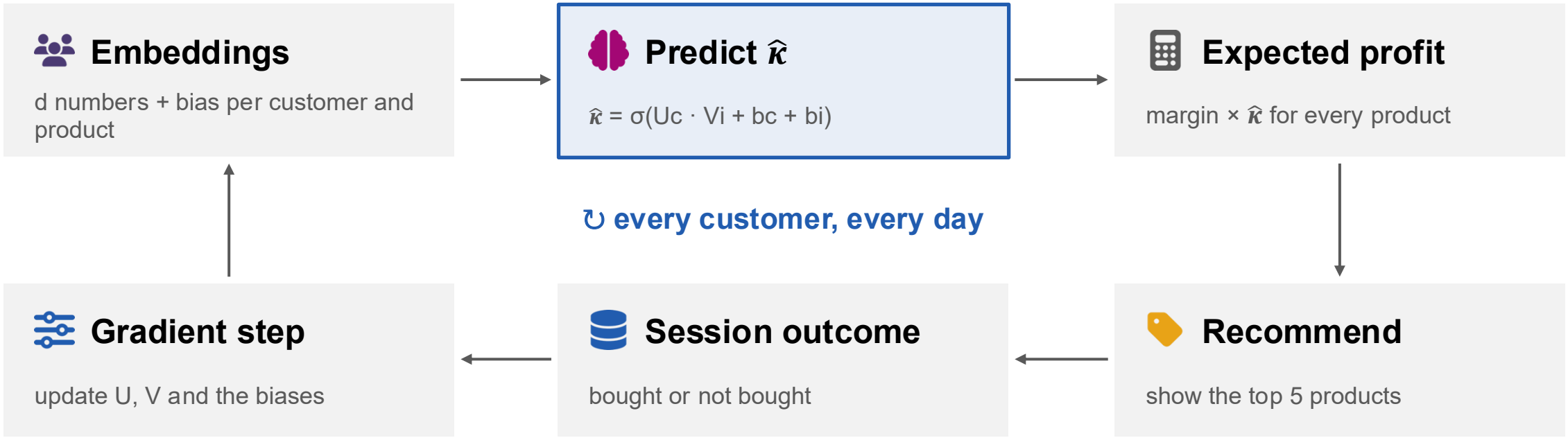
VR headset: \$170 × 0.88 = **\$150**. Silk scarf: \$174 × 0.23 = **\$40**. A high margin alone is not enough.

ETH zürich

# The agent: CF

Collaborative filtering by matrix factorization

# Collaborative filtering: the model




**Goal:** learn $\hat{\kappa}$ close to $\kappa$, also for products a customer has never seen, and show the most profitable 5.

# Learning, session by session

One session of one customer (example)

| | | | |
|---|---|---|---|
| VR Gaming Headset | bought | target 1 | $\hat{\kappa} \uparrow$ |
| Carbon Road Bicycle | bought | target 1 | $\hat{\kappa} \uparrow$ |
| 4K Drone | not bought | target 0 | $\hat{\kappa} \downarrow$ |
| Camera Lens | bought | target 1 | $\hat{\kappa} \uparrow$ |
| Designer Silk Scarf | not bought | target 0 | $\hat{\kappa} \downarrow$ |

**Gradient step on the logistic loss**

```
new embedding = old embedding + learning rate · w_purchase/w_neg ·(target − κ̂ )
for customer, product and biases
```

- **Bought**: target 1, weight `w_purchase`.
- **Shown, not bought**: target 0, weight `w_neg`.
- **Training**: 300 iterations, 15 products per customer
- **Test**: 100 days on all 50 products.

# Hyperparameters: your main lever

◎ TODO 6

You start from this (it does not run yet):

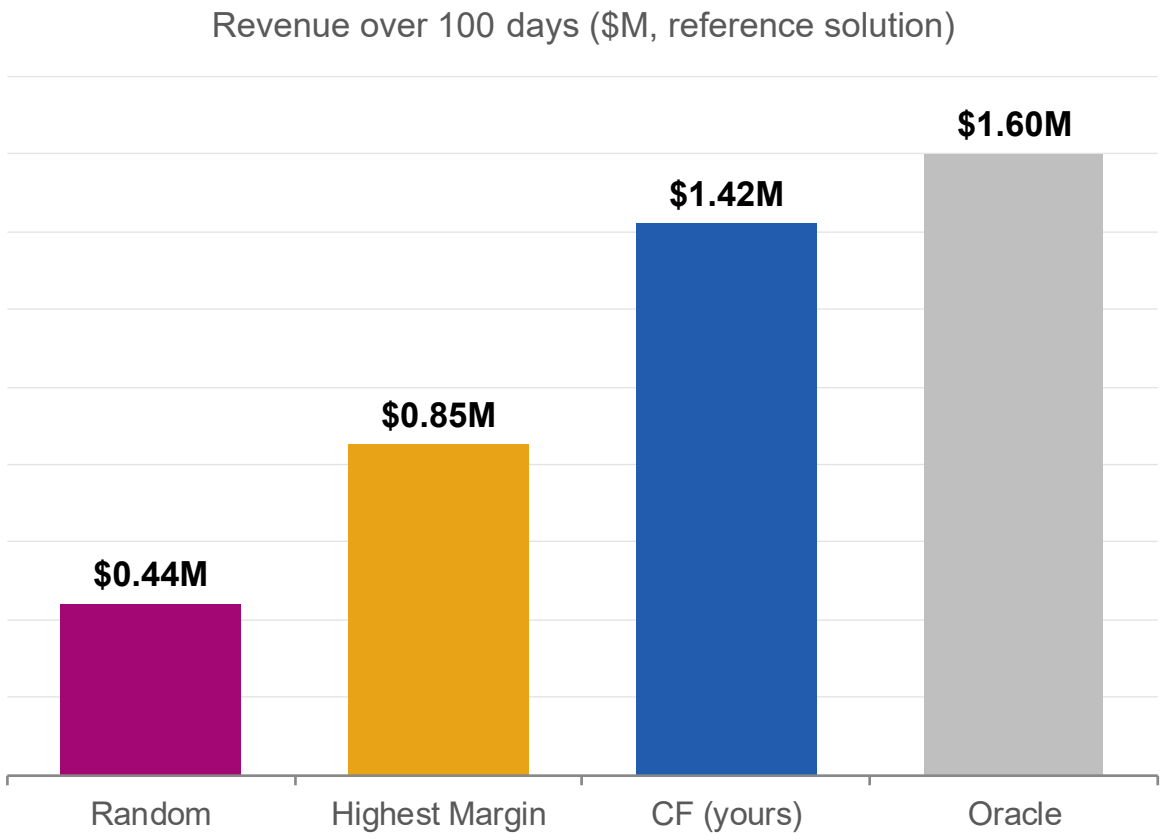
```
d           = None  # 2-20
lr          = None  # 0.01-0.3
reg         = None  # 1e-4-1e-1
w_purchase  = None  # 0.1-2.0
w_neg       = None  # 0.1-2.0
```

There is no single right answer. Judge your choice with the learning curves and the final test

**What each one does:**

- **d**: length of the embeddings. More detail vs. overfitting.
- **lr**: step size. Speed vs. stability
- **reg**: penalty on large weights. Overfitting vs. underfitting.
- **w_purchase / w_neg**: how strongly to learn from purchases and from rejections.
- Change **one at a time** and watch the validation curves.

# Test: 100 days against two baselines




**Random**
5 random products. Conversion ≈ 0.32.

**Highest Margin**
Always the 5 products with the highest margin. Conversion ≈ 0.27.

**CF (yours)**
Highest expected profit per customer. Conversion ≈ 0.65.

**Success:** CF > Highest Margin > Random.
Expect about +60–75% over Highest Margin.

ETH zürich

# Tasks

What you implement in the notebook

# TODOs: 6 tasks



SECTION 2.2

**Predict**

◎ TODO 1

$\hat{\kappa}$ = σ(Uc·Vi + bc + bi)

SECTION 2.3

**Recommend**

◎ TODO 2

margin × $\hat{\kappa}$

SECTION 2.4

**Learn**

◎ TODO 3

Targets for bought and not bought

SECTION 2.5

**Update**

◎ TODO 4

One gradient descent step

SECTION 2.6

**Check**

◎ TODO 5

Fix TODO 1–4 until all tests pass

SECTION 3.1

**Tune & test**

◎ TODO 6

Choose hyperparameters, train, beat the baselines

- **Basic Python implementations** with hints
- After TODO 1–4, a **check cell** tells you if it works.

```
✓ κ prediction: 0.6267 (valid range [0,1])
✓ All recommended items exist in catalogue
✓ κ prediction changed (learning working)

Your implementations appear to be working correctly.
```